# The Anomaly Guard

## Recommended project direction

Build a **local, explainable HTTP request anomaly detector** with a traffic simulator, a rule engine, a statistical engine, and an evaluation dashboard. The central experiment should be: **Does adding a learned normal-traffic baseline improve detection of held-out suspicious requests at an acceptable false-positive rate?**

Assume a university-level project implemented in Python. Keep the first version offline: process request records and replay them as a stream, rather than implementing a network sniffer or production firewall. Train the statistical baseline on legitimate requests; this is a novelty-detection setup, distinct from finding outliers in an already mixed training dataset.[^1]

**Scope boundary:** produce alerts and explanations, not claims that an intrusion succeeded. OWASP explicitly distinguishes suspicious validation failures from proof of attack.[^2]

All thresholds, dataset sizes, and schedules below are **proposed design choices**, not measured results.

## What the application contains

| Component | Required behavior | Main output |
|---|---|---|
| Traffic simulator | Generate legitimate, malicious-intent, and benign-invalid request scenarios; replay with controllable ordering and timestamps | Request records and separate ground-truth metadata |
| Parser | Interpret query parameters, form fields, and JSON bodies consistently | Structured fields and parsing flags |
| Feature extractor | Measure lengths, character composition, repetition, and structure | Numeric feature vector |
| Rule engine | Apply endpoint schemas and a small set of suspicious-pattern checks | Rule identifiers and severity |
| Baseline trainer | Learn expected behavior from legitimate training traffic | Versioned endpoint/field profiles |
| Statistical detector | Compare each request with the appropriate profile | Anomaly scores and supporting measurements |
| Decision engine | Combine rules and statistical evidence using validation-selected thresholds | Normal, review, or alert |
| Storage and dashboard | Display explanations, compare detectors, inspect mistakes, and export results | Searchable events and evaluation artifacts |

Use one shared detection library for the simulator, dashboard, and evaluation runner. This prevents the demonstration and the experiment from accidentally using different logic.

### Architecture and workflow

```
Simulator or JSONL import
          |
Bounded parsing and canonicalization
          |
Feature extraction
          |
     +----+----------------+
     |                     |
Rule engine         Statistical detector
     |                     |
     +----------+----------+
                |
          Decision engine
                |
       Event store and dashboard

Separate evaluation path:
Predictions + hidden ground truth -> metrics and error analysis

Separate training path:
Legitimate training records -> fitted baseline profiles
```

Implement the minimum useful interface as `fit(normal_records)` and `score(request)`. The scoring result should include a decision, component scores, triggered rules, explanations, baseline version, and processing time; ground truth must never enter this interface.

A demonstration should support **Train baseline**, **Replay traffic**, **Inspect alert**, and **Evaluate detectors**. Keep training explicit rather than updating the baseline automatically during every replay.

### Request representation

Use JSON Lines for request records. Store method, route, raw query string, selected headers, body text, timestamp, and a synthetic client identifier. Keep experiment labels, scenario identifiers, attack-family names, and generator-template identifiers in a separate metadata file joined by request ID.

For the MVP, model three small application routes:

| Route | Expected request | Useful distinctions |
|---|---|---|
| `GET /search` | Search text and optional numeric page | Short text versus long or structurally unusual queries |
| `POST /login` | Dummy username and password fields | Field structure and request size; never use real credentials |
| `POST /comment` | Free-form text or a small JSON object | Legitimate punctuation, Unicode, and longer text |

Apply size limits before parsing; preserve repeated query parameters instead of silently overwriting them. Keep raw and parsed representations for feature comparison, with a

documented decoding policy matching the mock application; OWASP recommends safe parsing, field-specific validation, and consistent normalization.[^3]

Do not repeatedly decode until a string stops changing. Treat malformed encodings as explicit parsing outcomes, and do not concatenate unrelated fields into a single string: an anomalous parameter should not disappear inside a large normal-looking body. Exclude multipart uploads and binary bodies from the first version.

### Features to extract

Start with a compact feature set, approximately 12–20 measurements, rather than a large machine-learning pipeline.

| Feature group | Proposed measurements | Interpretation to investigate |
|---|---|---|
| Length | Query bytes, body bytes, individual field length, longest field | Unusual size relative to endpoint and field |
| Character composition | Fractions of letters, digits, whitespace, quotes, brackets, slashes, and other categories | Distributional changes without requiring an exact attack string |
| Encoding | Percent-escape count and raw-to-decoded length difference | Unusual encoding use, subject to endpoint context |
| Repetition | Longest repeated-character run and distinct-character fraction | Highly repetitive or structurally unusual input |
| Entropy | Empirical character entropy | Additional description of character variability, not an attack label |
| Structure | Parameter count, repeated keys, unexpected fields, JSON parse failures | Deviations from the route's expected schema |

Define byte lengths and Unicode character counts separately. For the character model, choose one explicit representation—such as Unicode code-point categories on decoded field values—and use **mutually exclusive, exhaustive bins**. Do not mix overlapping counts such as “all punctuation” and “quotes” in one chi-square calculation.

Shannon entropy is a property of the observed character distribution, not a measure of malicious intent. Treat high and low entropy as context-dependent features; passwords, identifiers, and repeated ordinary text belong in benign tests too.[^4]

### Rule-based checks

Use two clearly separated types of rules:

- **Application-contract rules:** allowed method, required fields, numeric types, field length limits, permitted content types, and duplicate-key policy.
- **Suspicious-pattern heuristics:** a deliberately small collection of SQL-like, script-like, traversal-like, or control-character patterns, each with a rule ID and explanation.

Field-specific acceptance rules are preferable to globally banning punctuation; OWASP notes that apostrophes can be legitimate input. Accordingly, an apostrophe alone must not trigger a

high-severity alert, and pattern matching should not be presented as complete SQL-injection or XSS prevention.[^3]

Document each rule's intended purpose and known benign counterexamples. Keep schema violations distinguishable from malicious-pattern alerts so the evaluation can separate invalid input from attack intent.

## Statistical detection design

### Robust length outliers

For every supported endpoint and field, fit the median and median absolute deviation, or MAD, of legitimate training lengths. MAD and the interquartile range are less sensitive to extreme tails than standard deviation.[^5]

Define the robust score as:

$$m = \operatorname{median}(L_i), \qquad \operatorname{MAD} = \operatorname{median}(|L_i - m|)$$

$$z_{\text{robust}} = \frac{|L - m|}{1.4826 \operatorname{MAD}}$$

The scale factor makes MAD comparable to standard deviation under a normal reference distribution; it does not make HTTP lengths normally distributed. Select the operating threshold using held-out validation traffic, not an assumed universal cutoff.[^6]

If MAD is zero, handle the field explicitly with a documented tolerance or discrete-value policy; do not divide by an arbitrary tiny number and produce enormous scores. Mark missing or empty fields separately. An empirical upper-tail length threshold is an acceptable simpler alternative.

### Character-distribution differences

Learn baseline category probabilities from legitimate training field values. For a new value with length $n$, calculate observed category counts $O_i$ and expected counts $E_i = np_i$, then compute:

$$X^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

Pearson's goodness-of-fit test uses counts, requires matching observed and expected totals, and relies on adequate cell frequencies and a sampling model. Use smoothing to avoid zero expected probabilities, but do not claim that smoothing alone makes a sparse test valid.[^7]

For short inputs, merge rare categories using a policy fixed from training data, or skip this component with `insufficient_evidence`. SciPy documents the small-cell limitation; an unrestricted 256-byte test on short strings is therefore a poor default.[^7]

**Important statistical distinction:** natural-language and structured HTTP characters are dependent, and baseline probabilities are estimated. For the main detector, treat this as a **chi-**

**square-style discrepancy score**, calibrated against held-out legitimate requests in comparable length bands. Show theoretical p-values only as an optional teaching feature where assumptions are defensible, not as the probability that a request is malicious; p-values do not represent hypothesis probabilities.[^8]

For endpoint-level monitoring, optionally aggregate fixed-size windows and compare character or request-type distributions. Keep window alerts separate from individual-request alerts: a window-level change does not identify which request caused it.

### Intervals and uncertainty

Do not flag individual requests merely because they lie outside a confidence interval for the **mean** request length. A prediction interval concerns a future observation; a confidence interval concerns an estimated parameter.[^9]

Use empirical quantile thresholds for the initial request detector. Use confidence intervals where they add clearer value: report uncertainty around measured false-positive rates and recall; Wilson binomial intervals are available in statsmodels. If requests share sessions or templates, supplement these with grouped resampling rather than treating all near-duplicates as independent evidence.[^10]

### Optional multivariate detector

After the transparent baseline works, add Isolation Forest as an extension using the same numeric feature set. Scikit-learn supports anomaly scoring and threshold-based predictions, making it suitable as a comparison rather than a replacement for the statistical explanation.[^1]

Keep its training data, validation procedure, and test split identical to the simpler detectors. Do not add neural networks, multiple one-class models, or elaborate embeddings unless the basic experiment is already complete.

### Baselines and decisions

Create profiles by **HTTP method, route, content type, and field** where sufficient training support exists. For example, compare a comment with comments, not with login usernames. Payload-frequency modeling has historical precedent in PAYL, although this project should not inherit that paper's benchmark performance claims.[^11]

For sparse or unseen fields/routes, return `insufficient_baseline` and apply schema rules or a clearly labeled fallback profile. Avoid silently treating a novel endpoint as confirmed malicious.

A practical initial combination is:

- Convert each statistical component into an empirical anomaly percentile using legitimate calibration traffic.
- Aggregate components with a maximum or another fixed, documented function.
- Raise an alert for a high-severity rule or an aggregate statistical score above the alert threshold.

- Use a lower threshold for a review queue, and retain the evidence behind each outcome.

**Calibrate the complete decision**, including multiple fields and rule overrides, against the validation false-positive budget. Per-feature cutoffs do not automatically control the final request-level false-positive rate. If the rules alone exceed the chosen budget, revise the rules; raising only the statistical threshold cannot fix that.

Keep the design honest: an anomaly percentile describes unusualness relative to calibration traffic, **not attack probability**. An explanation should say “field length exceeds this profile's validated limit” rather than “99% certain attack.”

### Traffic and ground truth

Use controlled simulation as the main experiment because the assignment explicitly calls for simulated requests. A possible starting collection is **10,000 legitimate and 2,000 malicious-intent requests**, plus a separate benign-invalid set; these are planning quantities, not guarantees of adequate coverage.

| Scenario group | Cases to include | Evaluation purpose |
|---|---|---|
| Ordinary legitimate | Searches, logins with dummy credentials, comments, optional fields | Establish baseline behavior |
| Difficult legitimate | Apostrophes, accented names, emoji, multilingual text, long valid comments, code discussions | Expose false positives |
| Benign-invalid | Typing mistakes, invalid numeric fields, malformed client requests | Separate anomaly from malicious intent |
| Malicious-intent | SQL-injection-like, XSS-like, traversal-like, oversized, repeated, and encoded inputs | Measure detection on controlled attack scenarios |
| Hard malicious-intent | Short payloads and variants resembling legitimate lengths/composition | Expose false negatives |
| Benign distribution shift | A new client version, new language mix, or longer valid text | Test stability under changing normal behavior |

Run all scenarios against the local simulator or an explicitly authorized mock service. Labels describe the generator's intended scenario, not proof of exploit success. Include matched benign and malicious records with the same methods, headers, and endpoints so the detector cannot succeed through incidental generator fingerprints.

As an optional external check, use CSIC 2010 HTTP data. Its documentation describes automatically generated e-commerce traffic and includes unintentional invalid requests among anomalous examples, so its anomalous label must not be equated uncritically with malicious intent. Keep its results separate from the custom simulator and acknowledge its age and application-specific distribution.[^12]

## Evaluation methodology

### Split before fitting

Use three roles rather than one random train/test split:

1. **Training:** legitimate requests only, for fitting profiles and any preprocessing.
2. **Validation:** held-out legitimate traffic for calibration; a separate labeled development subset may support rule and threshold comparisons.
3. **Final test:** untouched legitimate, malicious-intent, and benign-invalid cases for the reported experiment.

Never learn profiles, length bands, feature selection, or normalization from the test set; this would leak information and inflate performance estimates. Keep near-duplicate requests and related template variants in the same partition; group-aware splitting supports this separation. [13][14]

Add one special held-out experiment in which an attack family or payload-template family is absent from development. This measures generalization to that held-out family, not a general ability to detect every unknown or zero-day attack. For streaming experiments, also respect chronology and keep windows from crossing split boundaries.

### Compare useful alternatives

| Detector configuration | Question answered |
|---|---|
| Rules only | What does the predefined baseline detect? |
| Length outliers only | How much signal comes from input size? |
| Character discrepancy only | Does distributional evidence add useful detection? |
| Combined statistics | Do statistical components complement one another? |
| Rules plus statistics | Is the hybrid worth its extra false positives and complexity? |
| Optional Isolation Forest | Does a multivariate model outperform transparent baselines? |

Choose thresholds on validation data and compare systems under a common operational false-positive constraint, for example a **proposed maximum of 1% on legitimate validation requests**. Report actual final-test false-positive rates even when they miss this target. Also show a threshold sweep rather than only a single favorable operating point.

### Measure the right outcomes

Treat malicious intent as positive for the primary intrusion-detection experiment, and report benign-invalid requests separately. Optionally add a second task where all invalid or malicious scenarios are labeled anomalous; name that task explicitly so the definitions do not get mixed.

Report true positives, false positives, true negatives, false negatives, precision, recall, F1, false-positive rate, and false-negative rate. Precision is the fraction of alerts that are positives; recall

is the fraction of positives detected.[^15]

$$\mathrm{FPR} = \frac{FP}{FP + TN}, \qquad \mathrm{FNR} = \frac{FN}{TP + FN}$$

Include average precision or a precision-recall curve for score-based detectors, per-family recall, and false-positive rates by endpoint and difficult-legitimate subgroup. Precision-recall analysis is particularly useful for imbalanced classification. Repeat the evaluation at several explicitly chosen simulated attack prevalences; do not present these as measured internet attack rates.[^15]

Measure median and 95th-percentile per-request scoring latency, throughput, and memory on a documented machine. Separate detector time from dashboard rendering, file I/O, and simulated waiting time.

Finally, inspect representative false positives and false negatives manually. For each mistake, record the relevant field, evidence, likely cause, and possible improvement without retuning against the final test results.

### Dashboard requirements

Keep five views:

- **Traffic:** replay controls, endpoint filters, request counts, processing status, and parsing failures.
- **Alert inspector:** safely rendered request text, rules, per-field scores, baseline measurements, and decision reasons.
- **Normal versus unusual:** observed feature distributions and baseline comparisons.
- **Evaluation:** confusion matrices, precision-recall curves, threshold trade-offs, per-family results, and detector comparisons.
- **Model settings:** profile coverage, training sample counts, version, thresholds, and explicit retraining action.

Show ground-truth labels only in experiment mode, never as an input to the detector. Display unsupported payloads, skipped tests, and missing profiles instead of hiding them behind a normal decision.

### Implementation structure

Recommended stack: **Python, NumPy/pandas, SciPy, Streamlit, SQLite or JSONL, and pytest.** Use scikit-learn only for metrics and the optional model; add FastAPI only if a real local ingestion endpoint is required. No separate frontend, message broker, or distributed infrastructure is necessary for the proposed scope.

```
anomaly_guard/
  app.py                 # dashboard
  cli.py                 # training, replay, evaluation
  core/
    schema.py
```

```
    parser.py
    features.py
    rules.py
    baseline.py
    statistics.py
    decision.py
  simulation/
    generator.py
    scenarios.py
  evaluation/
    splits.py
    metrics.py
    experiments.py
  config/
    routes.yaml
    rules.yaml
  tests/
  data/
  models/
  results/
  README.md
```

Persist feature definitions, decoding policy, category bins, training counts, profile statistics, calibration settings, and configuration hashes with each baseline. Keep the evaluation runner independent of the UI so a complete experiment is reproducible from a command.

## Build plan and deliverables

| Stage | Work | Completion evidence |
|---|---|---|
| Foundation | Define routes, request schema, labels, and simulator | Valid and invalid fixtures with reproducible seeds |
| Rules | Add bounded parsing, features, and explainable checks | Unit tests and a working rules-only baseline |
| Statistics | Fit per-field profiles; add length and composition scores | Verified handling of empty values, zero MAD, and sparse counts |
| Calibration | Implement split discipline and hybrid thresholds | Validation results with frozen settings |
| Interface | Add replay and alert inspection | Demonstration of both a detected attack scenario and a benign false alarm |
| Experiment | Run ablations, held-out-family tests, and resource measurements | Exported predictions, metrics, plots, and error analysis |

Allow roughly **four to six weeks as a planning estimate**, adjusting to team size and experience. Prioritize a complete rules/length/composition experiment over additional algorithms.

The final submission should contain source code, setup instructions, the simulator and data manifest, configuration files, baseline artifacts, automated tests, exported predictions and metrics, plots, and a short technical report describing methodology and limitations. Include a reproducible demonstration script and a clear explanation of what the detector cannot infer.

### Security and project limitations

Treat every imported payload as untrusted text: never execute it, render it as active HTML, or concatenate it into SQL. OWASP warns that unsafe logging can create injection risks and recommends structured logs and exclusion of credentials and tokens.[^2]

Enforce limits on body size, parsing depth, parameter count, regex work, and log growth; record oversized inputs without processing an unbounded body. Restrict the demo to localhost, use dummy secrets, and keep sensitive raw data out of persistent storage.[3][2]

The main limitations to document are simulator bias, incomplete normal-traffic coverage, endpoint cold starts, distribution drift, character-order blindness in simple histograms, and attacks that look statistically normal. Explain these as boundaries of the chosen representation and experiment, not reasons to promise a more complex model will solve them.

**Completion criterion:** a reproducible detector that explains its alerts and demonstrates, through fair comparisons, when statistics help, when they create false alarms, and which malicious-intent requests they miss. Improved detection is a hypothesis to test—not a result to assume.

### References


1. 2.7. Novelty and Outlier Detection - Scikit-learn - Outlier detection is similar to novelty detection in the sense that the goal is to separate a core o...
2. Logging - OWASP Cheat Sheet Series - Log Injection occurs when an application includes untrusted data in an application log message (e.g....
3. Input Validation - OWASP Cheat Sheet Series - Unicode Standard Annex #15 defines the normalization forms and warns that compatibility normalizatio...
4. entropy — SciPy v1.18.0 Manual - Shannon entropy quantifies the expected uncertainty inherent in the possible outcomes of a discrete ...
5. 1.3.5.6. Measures of Scale - Information Technology Laboratory - That is, confidence intervals based on the standard deviation tend to lack precision if the underlyi...
6. Consensus Mean - Information Technology Laboratory - NIST - The Mandel-Paule estimates are used as starting values for the consensus mean and between lab varian...
7. chisquare — SciPy v1.18.0 Manual - A typical rule is that all of the observed and expected frequencies should be at least 5. According ...
8. American Statistical Association Releases Statement on ... - P-values can indicate how incompatible the data are with a specified statistical model. P-values do ...
9. 4.1.3.2. Prediction - Information Technology Laboratory - This interval gives the range of plausible values for a single future pressure measurement observed ...
10. statsmodels.stats.proportion.proportion_confint¶ - Must be in (0, 1) - **method : {"normal", "agresti_coull", "beta", "wilson", "binom_test"}¶** defaul...
11. Anomalous Payload-Based Network Intrusion Detection - We present a payload-based anomaly detector, we call PAYL, for intrusion detection. PAYL models the ...

12. HTTP DATASET CSIC 2010 - petescully.co.uk - As it is a web application in Spanish, the data set contains some Latin characters. The dataset is g...
13. 12. Common pitfalls and recommended practices - scikit-learn - The purpose of this chapter is to illustrate some common pitfalls and anti-patterns that occur when ...
14. 3.1. Cross-validation: evaluating estimator performance - scikit-learn - GroupShuffleSplit provides a random sample (with replacement) of the train / test splits generated b...
15. Precision-Recall — scikit-learn 1.9.1 documentation - Example of Precision-Recall metric to evaluate classifier output quality. Precision-Recall is a usef...